



Sistema Web para Agendamiento de Citas Veterinarias

GUÍA DE EXPOSICIÓN DEL PROYECTO

Documento técnico que explica paso a paso cómo se construyó el proyecto PROVETCARE, incluyendo arquitectura, tecnologías, y flujos de datos.

Versión 1.0.0 - Enero 2026

■ Índice

- 1. Introducción al Proyecto
- 2. Stack Tecnológico
- 3. Arquitectura del Sistema
- 4. Fase 1: Configuración Inicial
- 5. Fase 2: Desarrollo del Backend
- 6. Fase 3: Desarrollo del Frontend
- 7. Fase 4: Funcionalidades Avanzadas
- 8. Fase 5: Seguridad
- 9. Flujo de Datos Completo
- 10. Demostración del Sistema

1. Introducción al Proyecto

¿Qué es PROVETCARE?

PROVETCARE es un **Sistema Web Completo** para la gestión de citas veterinarias, diseñado para reemplazar los métodos tradicionales (cuadernos y WhatsApp) con una solución digital moderna y eficiente.

Problema que Resuelve

Antes	Después
Citas agendadas por WhatsApp	Sistema de calendario en línea
Historiales en cuadernos	Base de datos centralizada
Recetas escritas a mano	PDFs generados automáticamente
Sin recordatorios	Emails automáticos 24h antes
Cobros informales	Sistema de facturación integrado

Usuarios del Sistema

- **Clientes:** Dueños de mascotas que agendan citas
- **Veterinarios:** Profesionales que atienden y documentan consultas
- **Administradores:** Control total del sistema

2. Stack Tecnológico

Tecnologías Utilizadas

Tecnología	Razón de Uso
React 18	Interfaz reactiva y componentizada
Vite	Desarrollo ultra-rápido (HMR instantáneo)
Express.js	Framework backend ligero y flexible
PostgreSQL	Base de datos relacional robusta
Socket.io	Chat en tiempo real
JWT	Autenticación segura stateless
TailwindCSS	Estilizado rápido con utilidades
PDFKit	Generación de recetas y facturas en PDF
Nodemailer	Envío de emails y recordatorios

Diagrama de Arquitectura

El sistema sigue una arquitectura de **tres capas**: Frontend (React) → Backend (Express) → Base de Datos (PostgreSQL). La comunicación se realiza mediante HTTP REST API y WebSockets para funcionalidades en tiempo real como el chat.

3. Arquitectura del Sistema

Patrón de Arquitectura: Cliente-Servidor con MVC

El proyecto utiliza el patrón **Modelo-Vista-Controlador (MVC)** en el backend, donde las Rutas definen los endpoints, los Controladores manejan la lógica de negocio, y el Modelo interactúa con PostgreSQL.

Estructura de Carpetas

Carpeta	Descripción	Archivos
client/src/pages/	Páginas/vistas principales	12 páginas
client/src/components/	Componentes reutilizables	7 componentes
client/src/context/	Estado global (AuthContext)	1 archivo
client/src/services/	Cliente HTTP (Axios)	1 archivo
server/routes/	Definición de endpoints	11 archivos
server/controllers/	Lógica de negocio	11 archivos
server/middleware/	Auth y validaciones	3 archivos
server/database/	Scripts SQL	9 archivos

4. Fase 1: Configuración Inicial

Paso 1.1: Inicialización del Proyecto

Se creó la estructura base del proyecto con dos carpetas principales: **client/** (frontend con Vite + React) y **server/** (backend con Express.js). Cada una tiene su propio package.json con dependencias específicas.

Paso 1.2: Dependencias del Backend

Paquete	Versión	Función
express	^4.18.2	Framework web principal
pg	^8.11.3	Cliente PostgreSQL
bcryptjs	^2.4.3	Hash de contraseñas
jsonwebtoken	^9.0.2	Tokens JWT
socket.io	^4.6.2	WebSockets
helmet	^7.1.0	Headers de seguridad
zod	^3.22.4	Validación de datos
pdfkit	^0.17.2	Generación de PDFs

Paso 1.3: Base de Datos PostgreSQL

Se diseñó un esquema relacional con las siguientes tablas principales: **users** (usuarios con roles), **pets** (mascotas), **appointments** (citas), **medical_records** (historiales médicos), **prescriptions** (recetas), **invoices** (facturas) y **payment_charges** (cobros pendientes).

5. Fase 2: Desarrollo del Backend

Servidor Principal (server.js)

El archivo **server.js** es el punto de entrada del backend. Configura los middlewares de seguridad (Helmet, CORS, Rate Limiting), parsea JSON, monta todas las rutas en `/api/*`, e inicializa Socket.io para el chat en tiempo real.

Patrón Rutas → Controladores

Cada módulo del sistema sigue el patrón de separación de responsabilidades:

- **Rutas (routes/)**: Definen los endpoints HTTP (GET, POST, PUT, DELETE)
- **Controladores (controllers/)**: Contienen la lógica de negocio
- **Middleware**: Validación de tokens y permisos

Endpoints Principales

Módulo	Endpoints	Descripción
Auth	<code>/api/auth/*</code>	Registro, login, verificación
Pets	<code>/api/pets/*</code>	CRUD de mascotas
Appointments	<code>/api/appointments/*</code>	Gestión de citas
Medical Records	<code>/api/medical-records/*</code>	Historiales médicos
Prescriptions	<code>/api/prescriptions/*</code>	Recetas con PDF
Billing	<code>/api/billing/*</code>	Cobros y pagos
Invoices	<code>/api/invoices/*</code>	Facturas en PDF

6. Fase 3: Desarrollo del Frontend

Punto de Entrada (main.jsx)

El archivo **main.jsx** inicializa React y envuelve la aplicación con los providers necesarios: **BrowserRouter** para navegación SPA, **AuthProvider** para estado global de autenticación, y **Toaster** para notificaciones.

Sistema de Rutas (App.jsx)

Define todas las rutas de la aplicación, separando rutas públicas (Login, Landing) de rutas protegidas (Dashboard, Pets, Calendar). Las rutas protegidas utilizan el componente **ProtectedRoute** que verifica la autenticación.

Páginas del Sistema

Página	Ruta	Función
LandingPage	/	Página de inicio pública
Login	/login	Autenticación de usuarios
Dashboard	/dashboard	Panel principal del usuario
Calendar	/calendar	Calendario de citas
Pets	/pets	Gestión de mascotas
MedicalHistory	/medical-history/:id	Historial de una mascota
Chat	/chat	Chat en tiempo real
PaymentsPage	/payments	Facturación y cobros

Context API para Autenticación

El **AuthContext** maneja el estado global del usuario logueado. Proporciona funciones como `login()`, `logout()`, `register()`, y expone el usuario actual a toda la aplicación sin necesidad de prop drilling.

7. Fase 4: Funcionalidades Avanzadas

7.1 Chat en Tiempo Real (Socket.io)

Implementamos un sistema de chat bidireccional usando **Socket.io**. El servidor escucha conexiones, agrupa usuarios en 'salas' de conversación, y retransmite mensajes instantáneamente. Los mensajes se persisten en la base de datos.

7.2 Generación de PDFs

Usamos **PDFKit** para generar documentos profesionales:

- **Recetas médicas:** Con información del paciente, veterinario y medicamentos
- **Facturas:** Detalle de servicios, impuestos y totales
- **Recibos de pago:** Confirmación de transacciones

7.3 Sistema de Recordatorios por Email

Usando **Nodemailer** y **node-cron**, el sistema envía automáticamente recordatorios por email 24 horas antes de cada cita. El job cron se ejecuta diariamente a las 9:00 AM.

7.4 Historial Médico Completo

Cada mascota tiene un historial médico detallado que incluye: diagnósticos, tratamientos, peso, temperatura, vacunas aplicadas y recetas generadas. Los veterinarios pueden crear nuevos registros directamente desde una cita.

8. Fase 5: Seguridad

Medidas de Seguridad Implementadas

Vulnerabilidad	Solución	Implementación
SQL Injection	Prepared Statements	pool.query(\$1, [value])
XSS	Escape automático	React escapa HTML por defecto
CSRF	CORS + JWT en headers	No cookies de sesión
Fuerza Bruta	Rate Limiting	100 req/15min por IP
Contraseñas	Bcrypt hashing	Cost factor 10
Headers inseguros	Helmet	Headers automáticos

Autenticación con JWT

El sistema utiliza **JSON Web Tokens** para autenticación stateless. Al hacer login, el servidor genera un token firmado con una clave secreta. Este token se envía en cada request subsecuente en el header Authorization y tiene una validez de 7 días.

Control de Acceso por Roles

Los usuarios tienen roles (cliente, veterinario, admin) que determinan sus permisos. El middleware **requireAdmin** o **requireVet** verifica el rol antes de permitir acceso a ciertas rutas protegidas.

9. Flujo de Datos Completo

Ejemplo: Proceso de Agendar una Cita

Este diagrama muestra cómo fluyen los datos desde que el usuario agenda una cita hasta que el administrador la ve en su panel:

- Paso 1 - Usuario:** Selecciona fecha/hora en el componente Calendar.jsx
- Paso 2 - Frontend:** Llama `api.post('/appointments', datos)`
- Paso 3 - Axios:** Agrega token JWT automáticamente mediante interceptor
- Paso 4 - Server:** Recibe request en `appointmentRoutes.js`
- Paso 5 - Middleware:** `authMiddleware` verifica el token JWT
- Paso 6 - Controller:** Valida datos con `Zod` e inserta en BD
- Paso 7 - BD:** Nueva fila en tabla 'appointments' con `status='pending'`
- Paso 8 - Socket.io:** Emite evento 'nueva_cita' a administradores
- Paso 9 - Frontend:** Muestra `toast.success('Cita agendada!')`
- Paso 10 - Admin:** Ve la nueva cita en tiempo real en su dashboard

10. Demostración del Sistema

Credenciales de Prueba

Rol	Email	Contraseña
Administrador	admin@provetcare.com	admin123
Cliente	juan.perez@email.com	cliente123

Flujo de Demostración Sugerido

1. Login como Cliente - Mostrar formulario y verificación JWT
2. Dashboard - Mostrar estadísticas personalizadas
3. Registro de Mascota - Crear nueva mascota con validación
4. Agendar Cita - Usar el calendario interactivo
5. Login como Admin - Demostrar cambio de rol
6. Aprobar Cita - Mostrar flujo de aprobación
7. Consulta Médica - Crear historial y generar receta PDF
8. Chat - Mostrar comunicación en tiempo real

Métricas del Proyecto

Métrica	Valor
Páginas Frontend	12
Componentes React	7
Controladores Backend	11
Endpoints API	~40
Tablas de BD	~10
Líneas de Código (aprox)	~15,000

PROVETCARE v1.0.0 - Sistema desarrollado con React, Express.js y PostgreSQL

- Cuidando a quienes cuidan a nuestras mascotas